Recommender Systems

1. Content based (CB)
2. Collaborative filtering (CF)
3. Latent factor based

- Lists of favorites, “essential” items
- List of “Top 10”, Most popular, New items
- Tailored to individual users

Utility Matrix:

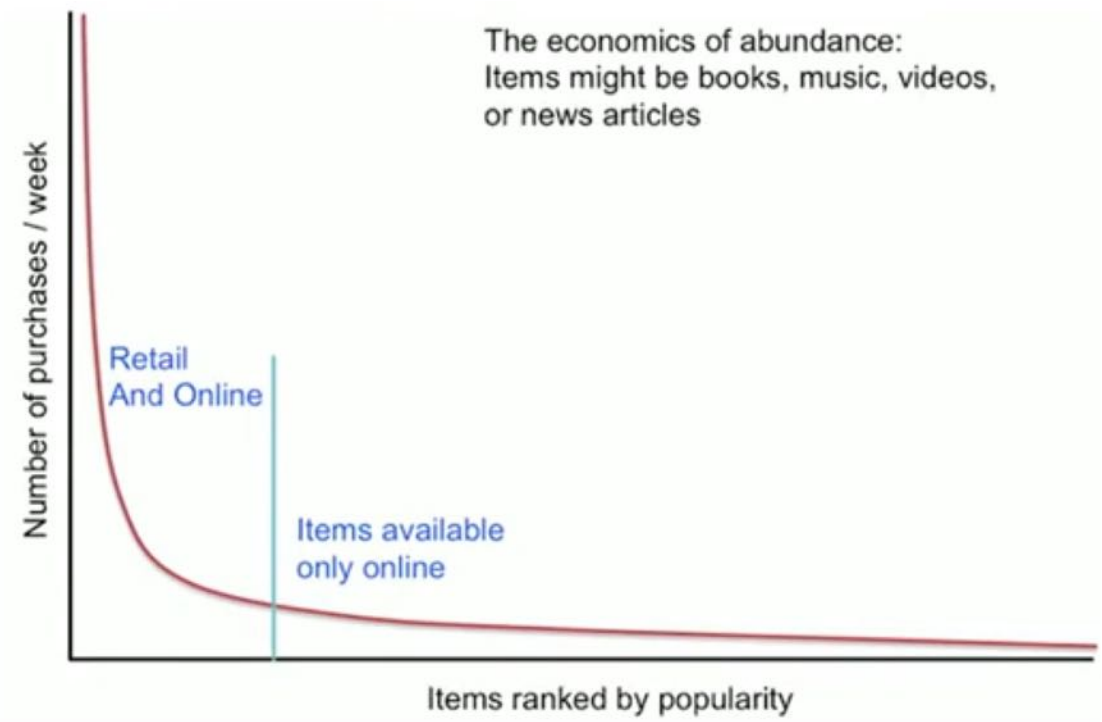
	Avatar	LOTR	Matrix	Pirates
Alice	1		0.2	
Bob		0.5		0.3
Carol	0.2		1	
David				0.4

Ratings:
- Explicit reviews and ratings
- Implicit from user buying behavior

Problem: Utility Matrix is very sparse, most people haven’t rated most items.
New items have no ratings, new users have no ratings

Long Tail

Total sales in long-tail may be big, but sales of each individual item are small and each item is almost invisible. Recommendations help.



Content-based Recommender Systems

Recommendations based on user history.
We build item profiles (vectors of features) and user profiles (vectors of feature preferences).
Then we match them (dot-product, angle between vectors)

For each item – profile, vector of features
Comedy, action, actor, director, etc.
How to extract most important features? Most important words/terms?
TF_IDF stands for **Term Frequency - Inverse Document Frequency**,
It is calculated by multiplying two numbers: **TF_IDF = TF * IDF**
where:
TF(t) = (Number of times term t appears in a document) / (Total number of terms in the document).
IDF(t) = log(Total number of documents / Number of documents with term t in it).
Doc profile – set of words with highest TF-IDF scores, together with their scores

User Profile:
user has rated … or user has bought …
Simple – take weighted average of items' features, normalize by average rating of the user.

Problem – will never recommend anything which doesn't fit user's history. What to recommend to a new user?

Collaborative filtering (CF)

CF is a technique used by recommender systems.
CF is a method of making automatic predictions (filtering) about the interests of a user by collecting preferences or taste information from many users (collaborating).

Typically, the workflow of a collaborative filtering system is:

- A user expresses his or her preferences by rating items (e.g. books, movies or CDs) of the system. These ratings can be viewed as an approximate representation of the user's interest in the corresponding domain.
- The system matches this user's ratings against other users' and finds the people with most "similar" tastes.
- With similar users, the system recommends items that the similar users have rated highly but not yet being rated by this user (presumably the absence of rating is often considered as the unfamiliarity of an item).

How to tell that users are similar?
Option 1 - The Jaccard index (a.k.a. “Intersection over Union” or “Jaccard similarity coefficient”), originally coined coefficient de communauté by Paul Jaccard.
Problem – it doesn’t account for the value of a rating.

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

Option 2 - Better approach – look at similarity of feature vectors (cosine similarity).
Problem here – how to deal with missing (not rated) components of vectors.

Option 3 – Centered cosine (**Pearson correlation**) – subtracting the row (user) mean to center around zero. Then missing data is treated as “average”. Also helps to resolve “tough rates vs easy rates” problem.

Collaborative filtering (CF) - continued

Once we have identified features for each movie,
and identified preferences for users, we can identify users similar to our user.

Then we can make predictions by calculating similarities between their items
and our user (and normalizing by sum of all their similarities). Then we take top-10 of
those weighted similarities.

This was user-to-user collaborative filtering.
We can also do item-to-item CF.
For each item we will find similar items, calculate similarity, use weighted similarity –
and select top 10.

Item-to-item similarity is simpler (less features, easier to understand) than user-to-user.
So in real life item-to-item outperforms user-to-user methods.

Collaborative filtering (CF) – continued - implementation

It is expensive (time consuming) to calculate similarity for all users and items.
The complexity is O(U), where U is the size of utility matrix = (Nusers * Nmovies).
Solutions:
- Near-neighbor search in high dimensions (LSH = Locality Sensitive Hashing, etc.)
- Clustering
- Dimensionality reduction

=======
Harry Potter effect (popularity bias) – recommending a popular item to many users makes it more popular.
=======
Combining item-to-item and user-to-user (Netflix).
Also adding other features (geography, language, age, etc.)
=======
Common practice:
- define similarity of items
- select “K” nearest neighbors for each pair (item, user)
- estimate ratings as base-line + CF piece (weighted normalized average)

=======
Testing algorithm: remove some data from Utility matrix – and try to predict it.
Compare actual data with predictions – using root-square measure.
Problems with this:
- prediction diversity (many similar items),
- prediction context (interest exists only during a trip),
- order of predictions

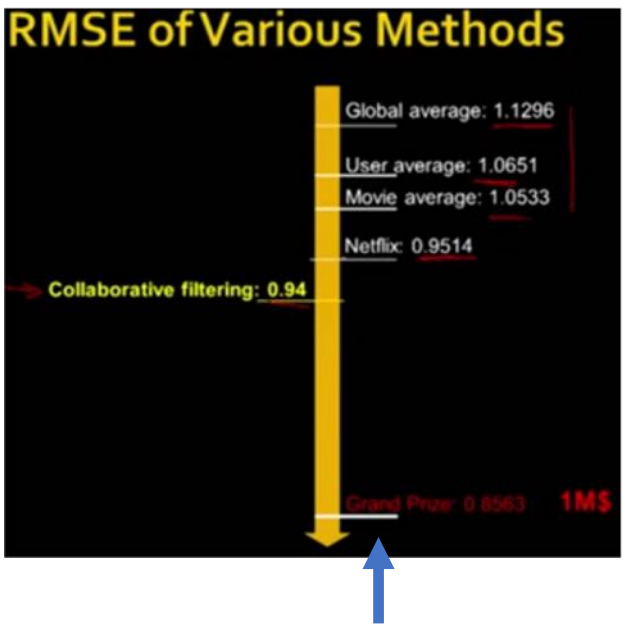
Possible solution – only compare several top ratings.

Latent Factor Recommender Systems

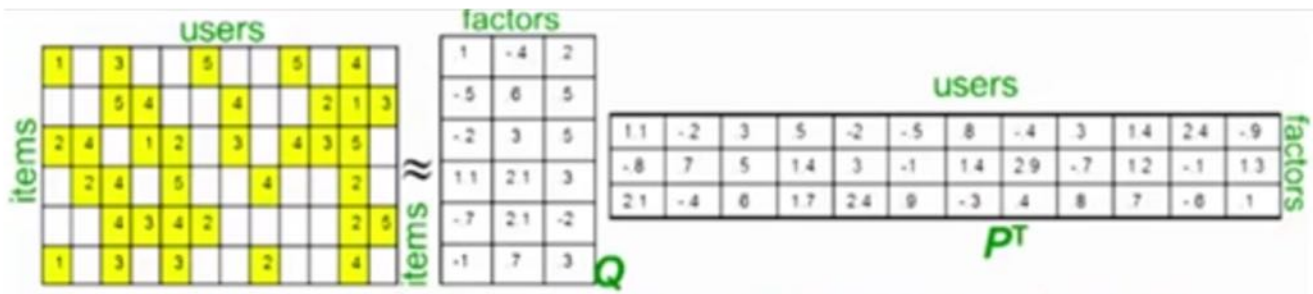
Each user is represented as a K-dimensional vector of latent variables.
Each movie is also represented as a K-dimensional vector (in picture below K=3)
We decompose original utility matrix into a product of 2 matrixes of latent vectors.
This is “matrix factorization”.
To find latent vectors, we solve minimization problem.

Dimensionality Reduction

If some rows/columns of a matrix can be presented as linear combinations of others, this decreases the “rank” of the matrix. We can then reduce the matrix to a smaller matrix (for example, convert from 3D to 2D plane).



Netflix prize (2009)



SVD isn’t defined when entries are missing!

- **Use specialized methods to find *P, Q*:**

$$\min_{P,Q} \sum_{(i,x)\in R} \left(r_{xi} - q_i \cdot p_x^T\right)^2$$

- **Note:**
  - We don’t require cols of ***P, Q*** to be orthogonal/unit length
  - ***P, Q*** map users/movies to a latent space
  - The most popular model among Netflix contestants

In Andrew Ng's Machine Learning course: Recommender System & Collaborative Filtering

we select n features (like romance, action, etc. – in 0..5 scale).

x - vector of features' values for a movie.

Θ - vector of features' values for user preferences.

y – matrix of ratings users give to movies (matrix size is (N_{users}, N_{movies}))

Given n-features, y & x , we can get Θ

Given n-features, y & Θ, we can get x

Or given "y", we can solve minimization problem against both x & Θ (and find features).

Note: after you've learned features, it's often very hard to come in and apply a human understandable metric to what those features are.

Andrew uses a (low rank) matrix factorization algorithm.

It is similar to the Singular Value Decomposition (SVD = diagonalization / rotation) of "y".

You're trying to compute Θ and X so that ($\Theta^T X - y$) is minimal in a sum of squares sense, subject to a regularization condition and ignoring any values that aren't in y.

The number of features "n" that you elect to use can be seen as corresponding to the number of singular values you'd include if you were reducing the dimensionality of your data using a SVD. If you didn't have any missing data and there were no regularization, these notions would correspond exactly, except that SVD gives you $USV^T=y$, where S is diagonal so the S factors would be smushed into the Θ and X matrices you computed.

Mean Normalization:

Calculate average rating for each movie, and subtract them from ratings for this movie in matrix "y". If we run our predictive algorithm for a user with no activity – it will predict zeros for all movies. But then we can add the average value for each movie – and recommend the ones with higher averages.

Amazon has released their recommendation engine as open source:
- https://github.com/amzn/amazon-dsstne

Amazon DSSTNE: Deep Scalable Sparse Tensor Network Engine

DSSTNE (pronounced "Destiny") is an open source software library
for training and deploying recommendation models
with sparse inputs, fully connected hidden layers, and sparse outputs.

Models with weight matrices that are too large for a single GPU
can still be trained on a single host.
DSSTNE has been used at Amazon to generate personalized product
recommendations for our customers at Amazon's scale.

It is designed for production deployment of real-world applications
which need to emphasize speed and scale over experimental flexibility.

Misc:

deFinetti Theorem: Exchangeable random variable can be rewritten as product of independent variables conditioned on some parameter (root cause).

Aldous Hoover Factorization, variants

Bias (user bias, product bias)

Ratings are not random. Distribution is not even.
It may be bi-modal (love/hate), or skewed to one side.

List of recommender systems:
- https://github.com/grahamjenson/list_of_recommender_systems

The Top 98 Recommender System Open Source Projects
- https://awesomeopensource.com/projects/recommender-system

The Top 19 Recommendation Engine Open Source Projects
- https://awesomeopensource.com/projects/recommendation-engine

Open Source Recommender Systems
- https://mathsgee.com/docs/recommender-systems/open-source-recommender-systems/

Recommendation System in Python
- https://towardsdatascience.com/recommendation-system-in-python-lightfm-61c85010ce17

Microsoft – Recommenders:
- https://github.com/microsoft/recommenders

10 Open-Source Datasets One Must Know To Build Recommender Systems:
- https://analyticsindiamag.com/10-open-source-datasets-one-must-know-to-build-recommender-systems/

NVIDIA Course on Building Recommender Systems:
- https://blogs.nvidia.com/blog/2020/05/14/whats-a-recommender-system/

Recommender Systems: Matrix Factorization using PyTorch
- https://towardsdatascience.com/recommender-systems-matrix-factorization-using-pytorch-bd52f46aa199

Building a strong baseline recommender using PyTorch, on a regular laptop
- https://towardsdatascience.com/how-to-build-a-strong-baseline-recommender-using-pytorch-on-a-regular-laptop-2ad497504fe6

Recommendation System Implementation With Deep Learning and PyTorch
- https://medium.com/swlh/recommendation-system-implementation-with-deep-learning-and-pytorch-a03ee84a96f4

Building an Incremental Recommender System
https://towardsdatascience.com/building-an-incremental-recommender-system-8836e30afaef

Amazon DSSTNE: Deep Scalable Sparse Tensor Network Engine:
https://github.com/amazon-archives/amazon-dsstne

==========

Amazon Personalize

https://aws.amazon.com/personalize/

Pros of **Amazon Personalize** :
- it is distributed and scalable, it works both in batch mode and real-time, you won't need heavy engineering
- it's feature-rich in terms of ML: contextual recommendations, cold-start solved, etc.
- it's from Amazon. They have some experience with recommendations